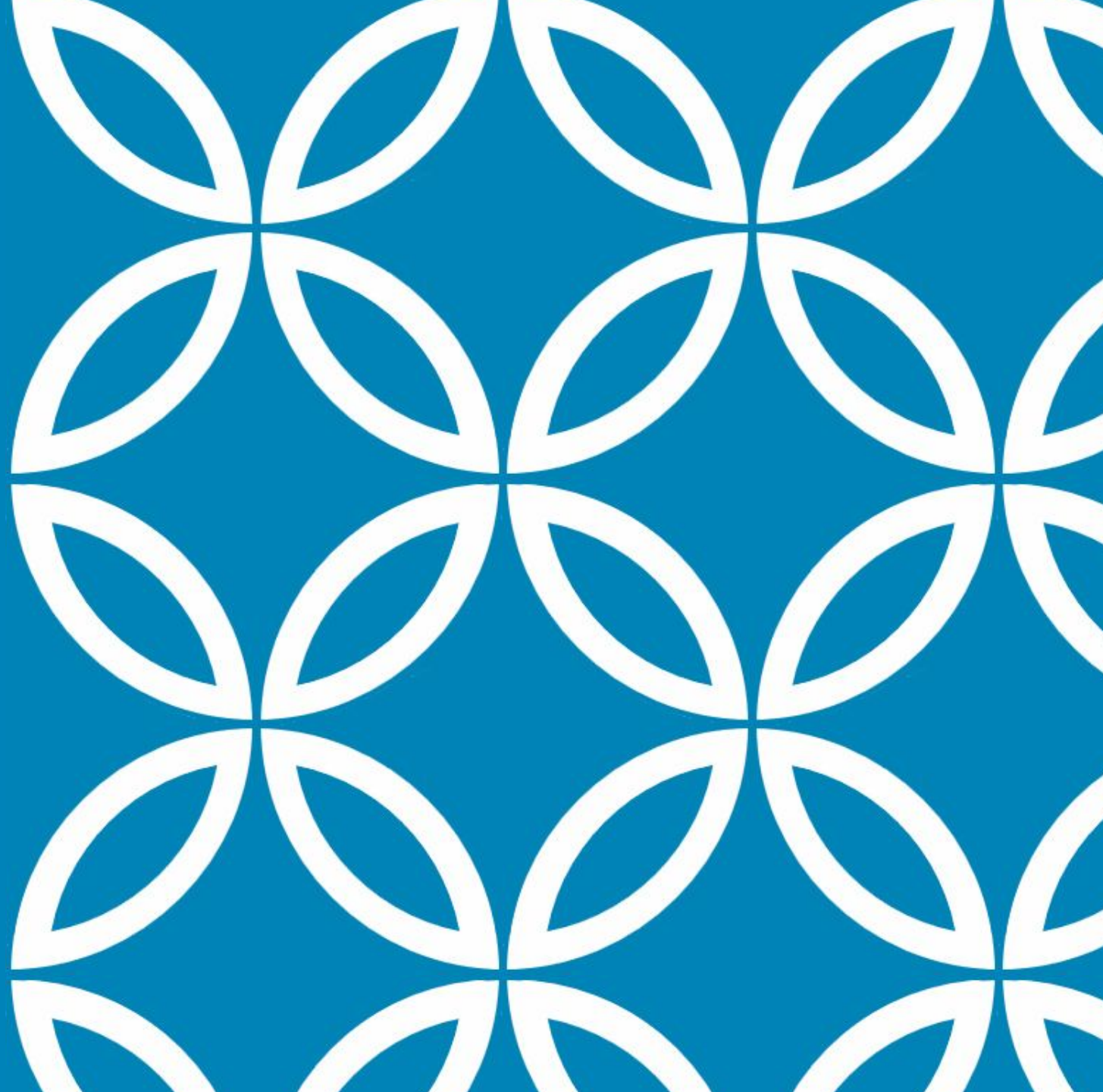


LINEAR REGRESSION

DR.BHAKTI PALKAR



WHAT IS LINEAR REGRESSION? ON?

Linear regression is one of the easiest and most popular Machine Learning algorithms.

Linear regression makes predictions for continuous/real or numeric variables

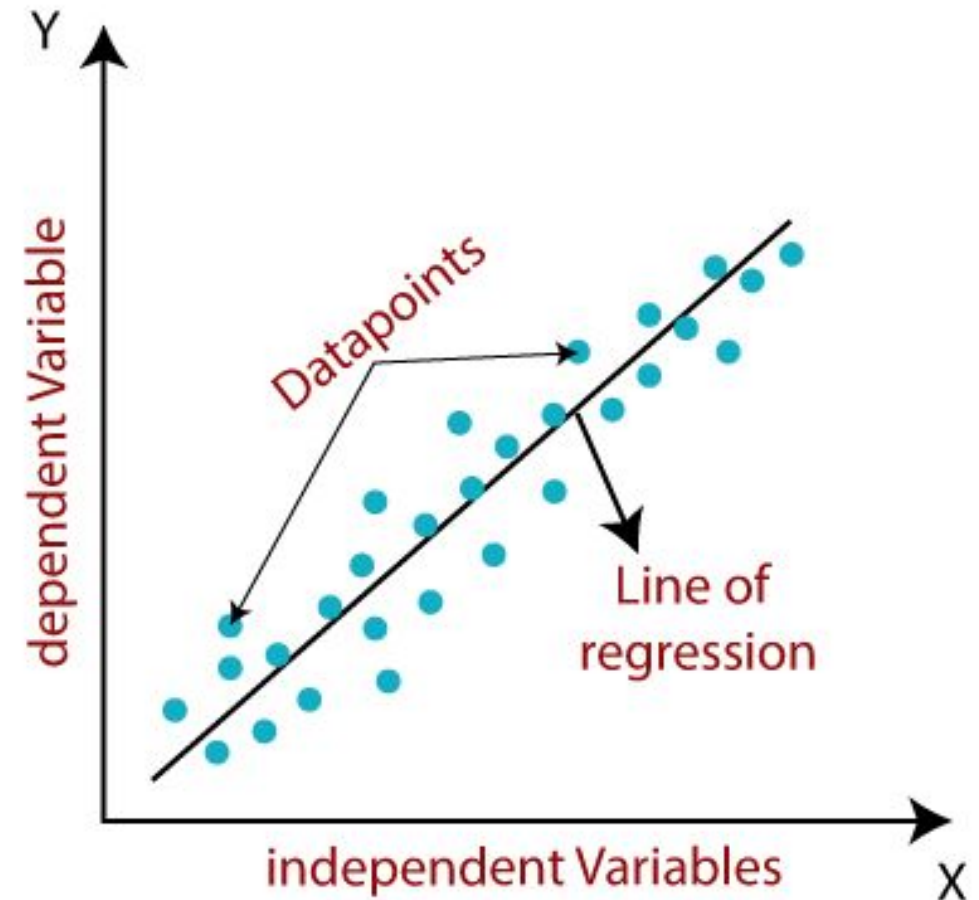
Three main uses of regression analysis are:

1. Predicting share price
2. Analyzing the impact of price changes

LINEAR REGRESSION

Linear regression shows the linear relationship, which means it finds how the value of the dependent variable (y) is changing according to the value of the independent variable (x).

The linear regression model provides a sloped straight line representing the relationship between the variables.



LINEAR REGRESSION

Mathematically, we can represent a linear regression as:

$$y = mx + c$$

$$y = a_0 + a_1x + \epsilon$$

Here,

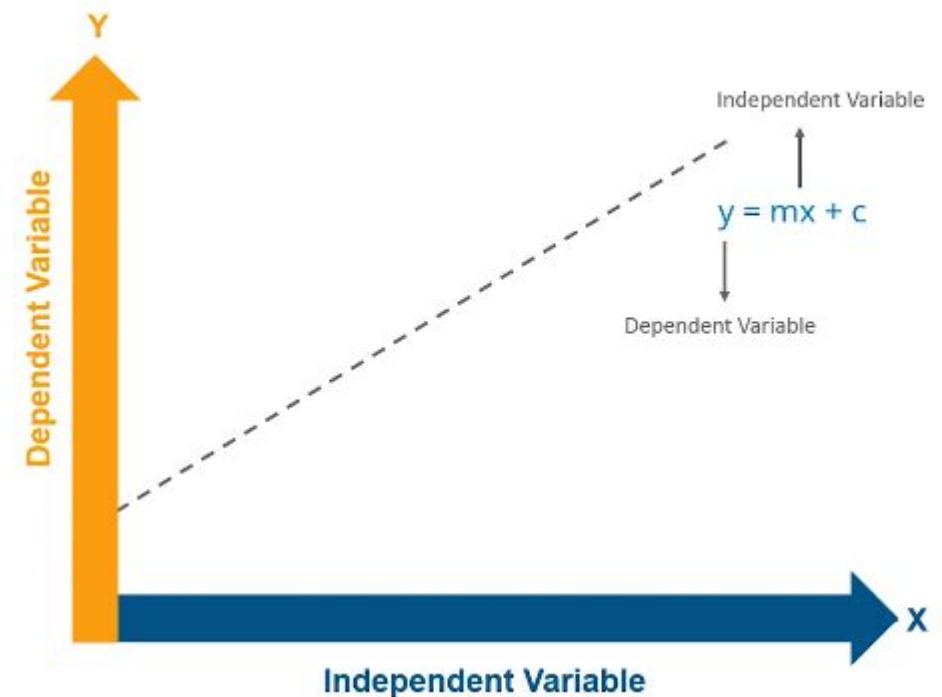
Y= Dependent Variable (Target Variable)

X= Independent Variable (predictor Variable)

a_0 = intercept of the line (Gives an additional degree of freedom)

a_1 = Linear regression coefficient (scale factor to each input value).

ϵ = random error



TYPES OF LINEAR REGRESSION

Simple Linear Regression:

If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

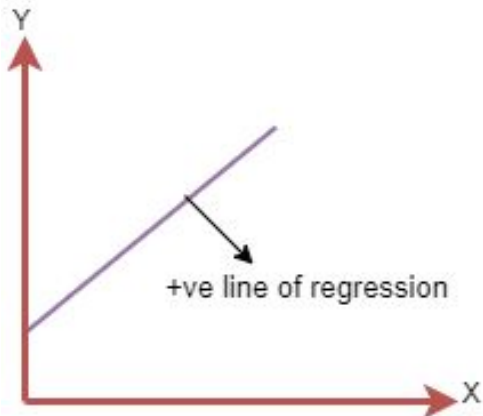
Multiple Linear regression:

If more than one independent variables are used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

LINEAR REGRESSION LINE

Positive Linear Relationship:

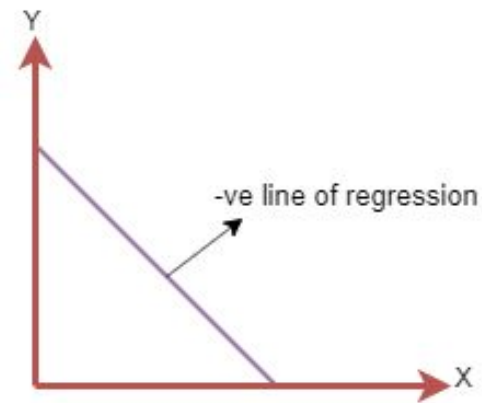
If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.



The line equation will be: $Y = a_0 + a_1X$

Negative Linear Relationship:

If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.



The line of equation will be: $Y = -a_0 + a_1X$

COST FUNCTION

Goal: find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

Loss: Error in our predicted value of **m** and **c**. Our goal is to minimize this error to obtain the most accurate value of **m** and **c**.

We will use the Mean Squared Error function to calculate the loss. There are three steps in this function:

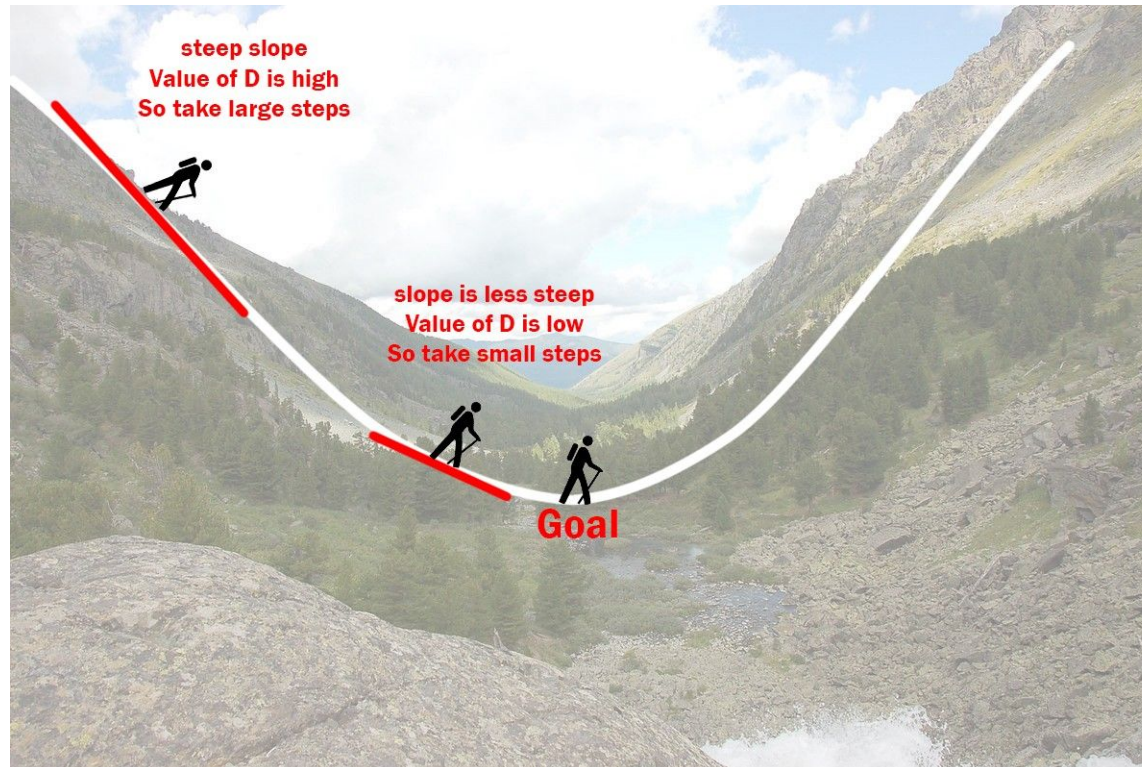
1. Find the difference between the actual y and predicted y value($y = mx + c$), for a given x.
2. Square this difference.

3. $E = \frac{1}{n} \sum_{i=0}^n (y_i - \bar{y}_i)^2$ ares for every value in X.

$$E = \frac{1}{n} \sum_{i=0}^n (y_i - (mx_i + c))^2$$

GRADIENT DESCENT ALGORITHM

Gradient descent is an iterative optimization algorithm to find the minimum of a function. Here that function is our Loss Function.



GRADIENT DESCENT ALGORITHM

Let's try applying gradient descent to $\mathbf{m}(\mathbf{a1})$ and $\mathbf{c}(\mathbf{a0})$ and approach it step by step:

1. Initially let $a_0 = 0$ and $a_1 = 0$. Let α be our learning rate. This controls how much the values of a_0 and a_1 change with each step. α could be a small value like 0.0001 for good accuracy.

2. Calculate the partial derivative of the loss function with respect to a_0 and a_1

$$J = \frac{1}{n} \sum_{i=1}^n (pred_i - y_i)^2$$

$$J = \frac{1}{n} \sum_{i=1}^n (a_0 + a_1 \cdot x_i - y_i)^2$$

$$\frac{\partial J}{\partial a_0} = \frac{2}{n} \sum_{i=1}^n (a_0 + a_1 \cdot x_i - y_i) \implies \frac{\partial J}{\partial a_0} = \frac{2}{n} \sum_{i=1}^n (pred_i - y_i)$$

$$\frac{\partial J}{\partial a_1} = \frac{2}{n} \sum_{i=1}^n (a_0 + a_1 \cdot x_i - y_i) \cdot x_i \implies \frac{\partial J}{\partial a_1} = \frac{2}{n} \sum_{i=1}^n (pred_i - y_i) \cdot x_i$$

GRADIENT DESCENT ALGORITHM

3. Now we update the current value of **a0** and **a1** using the following equation:

$$a_0 = a_0 - \alpha \times D_{a_0}$$

$$a_1 = a_1 - \alpha \times D_{a_1}$$

$$a_0 = a_0 - \alpha \cdot \frac{2}{n} \sum_{i=1}^n (pred_i - y_i)$$
$$a_1 = a_1 - \alpha \cdot \frac{2}{n} \sum_{i=1}^n (pred_i - y_i) \cdot x_i$$

4. We repeat this process until our loss function is a very small value or ideally 0 (which means 0 error or 100% accuracy). The value of **a0** and **a1** that we are left with now will be the optimum values.

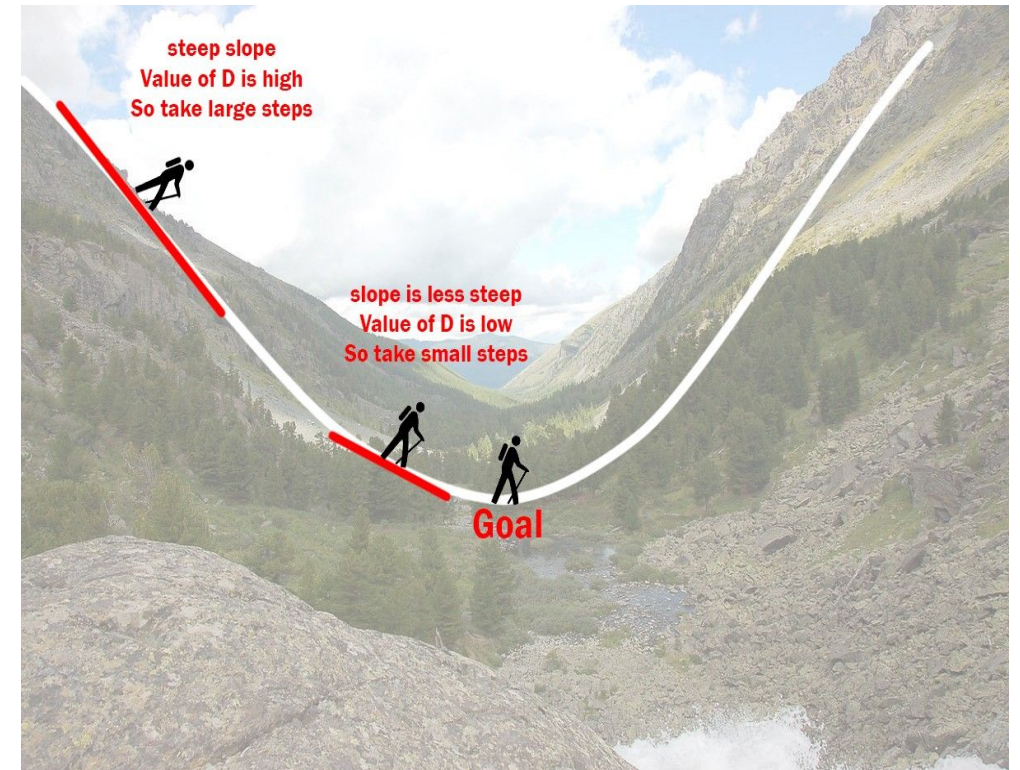
GRADIENT DESCENT ALGORITHM

Now going back to our analogy, a_0, a_1 can be considered the current position of the person. D is equivalent to the steepness of the slope and α (**learning rate**) can be the speed with which he moves.

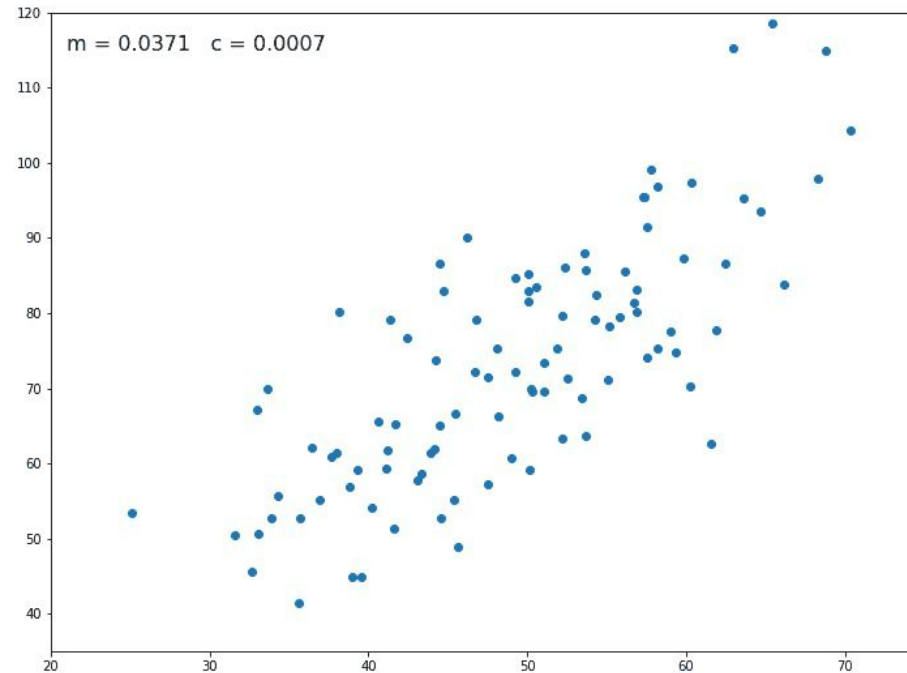
Now the new value of (a_0, a_1) that we calculate using the above equation will be his next position, and $\alpha \times D$ will be the size of the steps he will take.

When the slope is more steep (D is more) he takes longer steps and when it is less steep (D is less), he takes smaller steps.

Finally he arrives at the bottom of the valley which corresponds to our $\text{loss} = 0$.
Now with the optimum value of a_0 and a_1 our model is ready to make predictions !



THE VALUES OF M AND C ARE
UPDATED AT EACH ITERATION
TO GET THE OPTIMAL
SOLUTION



GOODNESS OF FIT

The Goodness of fit determines how the line of regression fits the set of observations.

R-squared method:

R-squared is a statistical method that determines the goodness of fit.

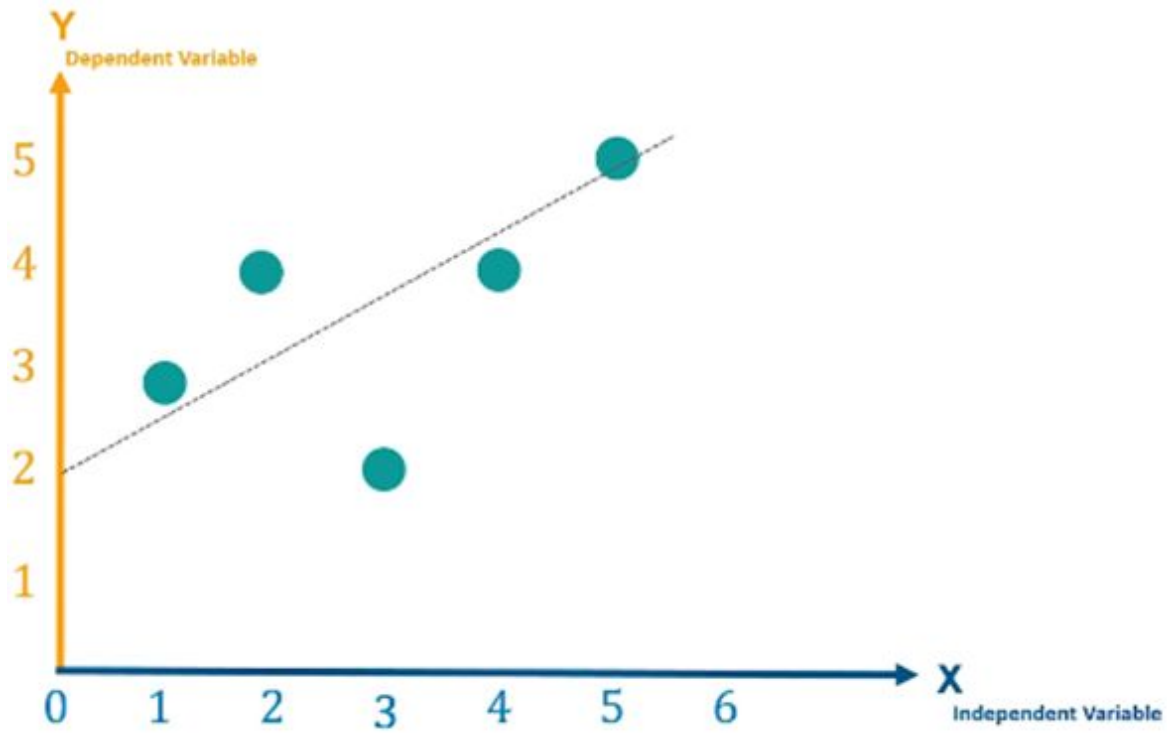
It measures the strength of the relationship between the dependent and independent variables on a scale of 0-100%.

The high value of R-square determines the less difference between the predicted values and actual values and hence represents a good model.

It is also called a **coefficient of determination**, or **coefficient of multiple determination**.
$$R\text{-squared} = \frac{\text{Explained variation}}{\text{Total Variation}}$$

Formula:

EXAMPLE



$X=[1,2,3,4,5]$
 $Y=[3,4,2,4,5]$

X	Y
1	3
2	4
3	2
4	4
5	5

Epoch 1:

$m = 0$ #a1, $c = 0$ #a0, $L = 0.0001$

$Y_{pred} = m * X + c$ # The current predicted value of Y

$$Y_{pred} = 0 \times [1, 2, 3, 4, 5] + 0$$

$$Y_{pred} = [0 \ 0 \ 0 \ 0 \ 0]$$

$D_m = (2/n) * \text{sum}(X * (Y_{pred} - Y))$ # Derivative wrt m

$$D_m = \left(\frac{2}{n}\right) \times \sum([1 \ 2 \ 3 \ 4 \ 5] \times ([0 \ 0 \ 0 \ 0 \ 0] - [3 \ 4 \ 2 \ 4 \ 5]))$$

$$D_m = -23.2$$

$D_c = (2/n) * \text{sum}(Y_{pred} - Y)$ # Derivative wrt c

$$D_c = \left(\frac{2}{n}\right) \times \sum([0 \ 0 \ 0 \ 0 \ 0] - [3 \ 4 \ 2 \ 4 \ 5])$$

$$D_c = -7.2$$

$m = m - L * D_m$ # Update m $m = 0 - 0.0001 \times -23.2$, $m = 0.002320000000000000004$

$c = c - L * D_c$ # Update c $c = 0 - 0.0001 \times -7.2$, $c = 0.00072$

Epoch 2:

$m = 0.00232000000000000004$ #a1, $c = 0.00072$ #a0, $L = 0.0001$

$Y_{pred} = m * X + c$ # The current predicted value of Y

$$Y_{pred} = 0.00232 \times [1, 2, 3, 4, 5] + 0.00072$$

$$Y_{pred} = [0.00304 \quad 0.00536 \quad 0.00768 \quad 0.01 \quad 0.01232]$$

$D_m = (2/n) * \text{sum}(X * (Y_{pred} - Y))$ # Derivative wrt m

$$D_m = \left(\frac{2}{n}\right) \times \sum([1 \ 2 \ 3 \ 4 \ 5] \times ([0.00304 \quad 0.00536 \quad 0.00768 \quad 0.01 \quad 0.01232] - [3 \ 4 \ 2 \ 4 \ 5]))$$

$$D_m = -23.1446400000000003$$

$D_c = (2/n) * \text{sum}(Y_{pred} - Y)$ # Derivative wrt c

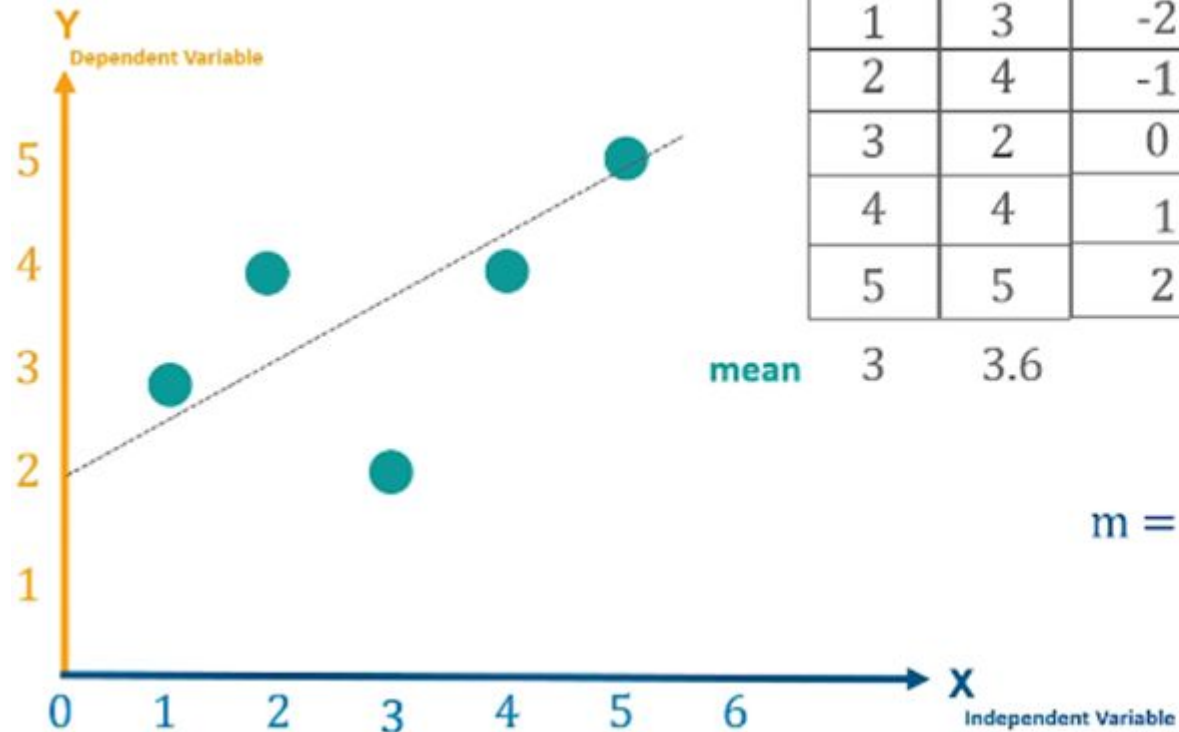
$$D_c = \left(\frac{2}{n}\right) \times \sum([0.00304 \quad 0.00536 \quad 0.00768 \quad 0.01 \quad 0.01232] - [3 \ 4 \ 2 \ 4 \ 5])$$

$$D_c = -7.1846400000000001$$

$m = m - L * D_m$ # Update m $m = 0.00232 - 0.0001 \times -23.144$, $m = 0.00463$

$c = c - L * D_c$ # Update c $c = 0.00072 - 0.0001 \times -7.1846400000000001$, $c = 0.00143$

EXAMPLE



x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})^2$	$(x - \bar{x})(y - \bar{y})$
1	3	-2	-0.6	4	1.2
2	4	-1	0.4	1	-0.4
3	2	0	-1.6	0	0
4	4	1	0.4	1	0.4
5	5	2	1.4	4	2.8
mean		3	3.6	$\Sigma = 10$	$\Sigma = 4$

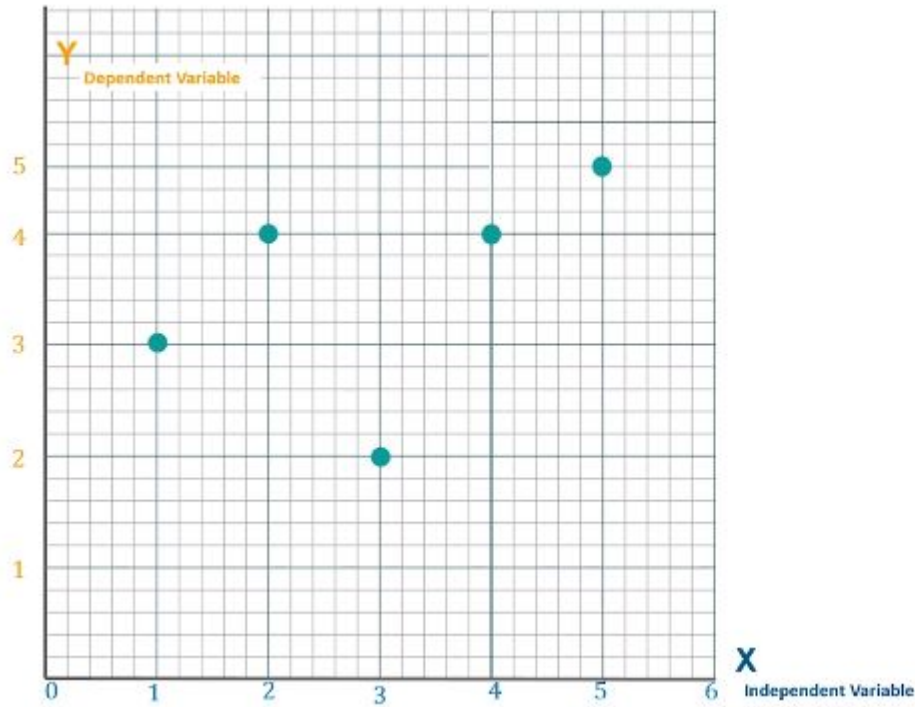
$$m = \frac{\Sigma (x - \bar{x})(y - \bar{y})}{\Sigma (x - \bar{x})^2} = \frac{4}{10}$$

$y = mx + c$
 $3.6 = 0.4 \cdot 3 + c$
 $3.6 = 1.2 + c$
 $c = 2.4$

$$y = m \cdot x + c$$

$$y = 0.4 \cdot x + 2.4$$

MEAN SQUARE ERROR



$$m = 0.4$$

$$c = 2.4$$

$$y = 0.4x + 2.4$$

For given $m = 0.4$ & $c = 2.4$, lets
predict values for y for $x = \{1, 2, 3, 4, 5\}$

$$y = 0.4 \times 1 + 2.4 = 2.8$$

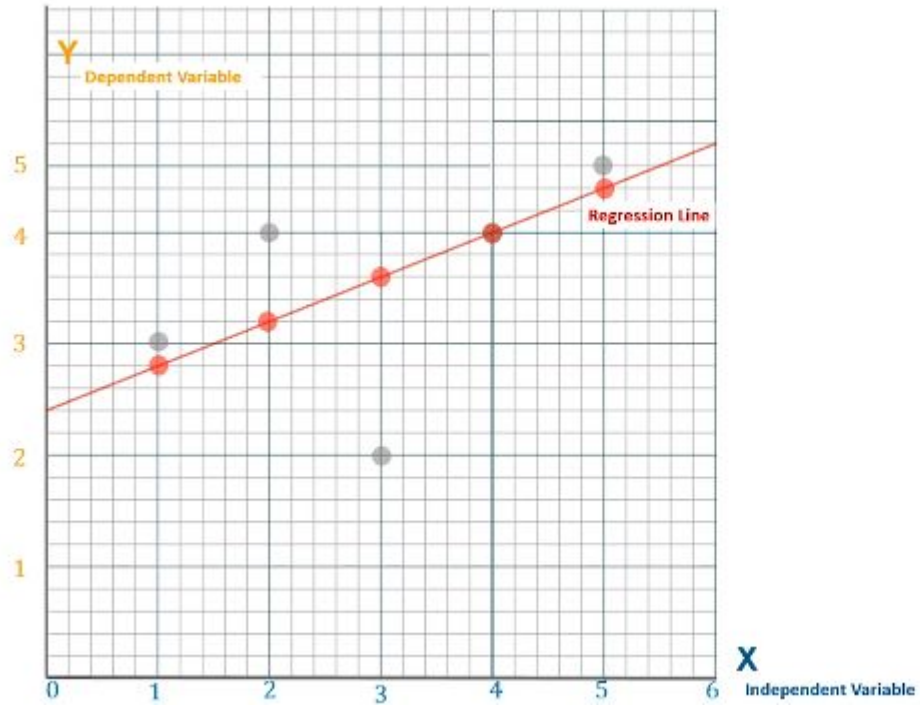
$$y = 0.4 \times 2 + 2.4 = 3.2$$

$$y = 0.4 \times 3 + 2.4 = 3.6$$

$$y = 0.4 \times 4 + 2.4 = 4.0$$

$$y = 0.4 \times 5 + 2.4 = 4.4$$

MEAN SQUARE ERROR



$$m = 0.4$$

$$c = 2.4$$

$$y = 0.4x + 2.4$$

For given $m = 0.4$ & $c = 2.4$, lets
predict values for y for $x = \{1, 2, 3, 4, 5\}$

$$y = 0.4 \times 1 + 2.4 = 2.8$$

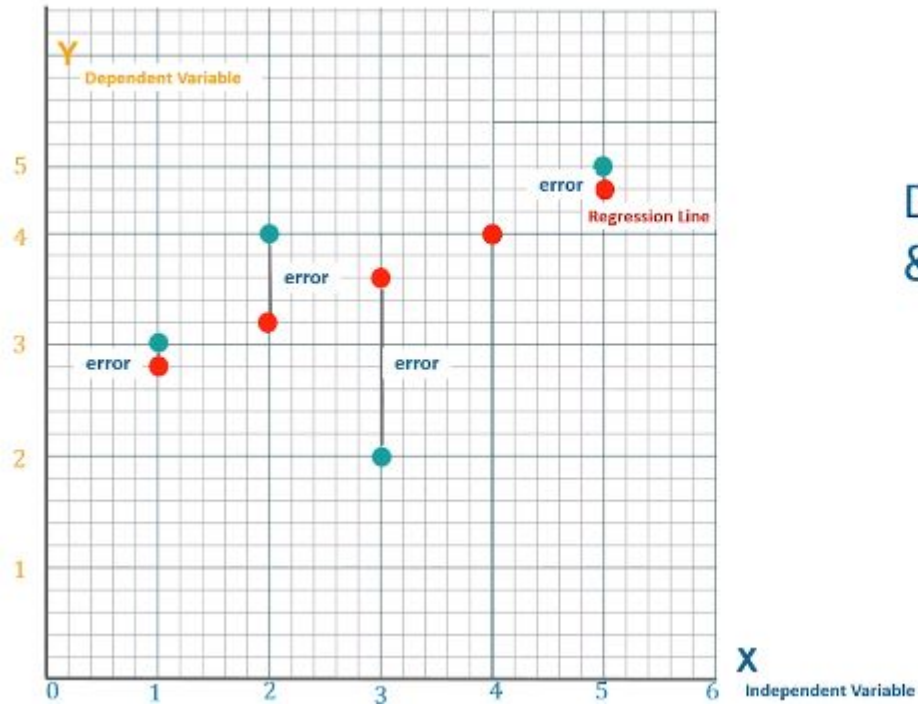
$$y = 0.4 \times 2 + 2.4 = 3.2$$

$$y = 0.4 \times 3 + 2.4 = 3.6$$

$$y = 0.4 \times 4 + 2.4 = 4.0$$

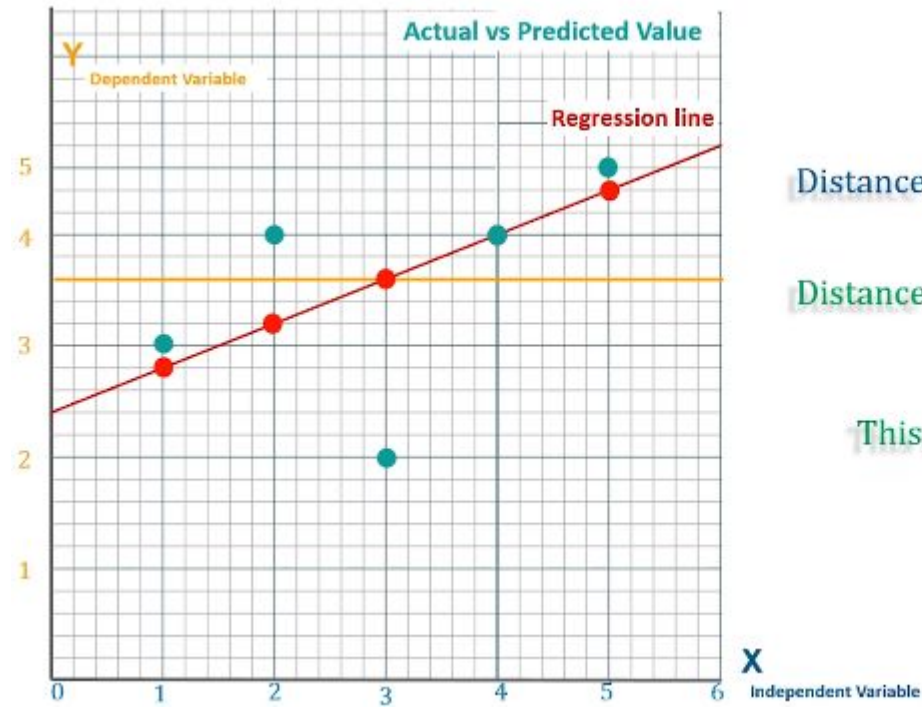
$$y = 0.4 \times 5 + 2.4 = 4.4$$

MEAN SQUARE ERROR



Distance between actual
& predicted value

GOODNESS OF FIT-R²



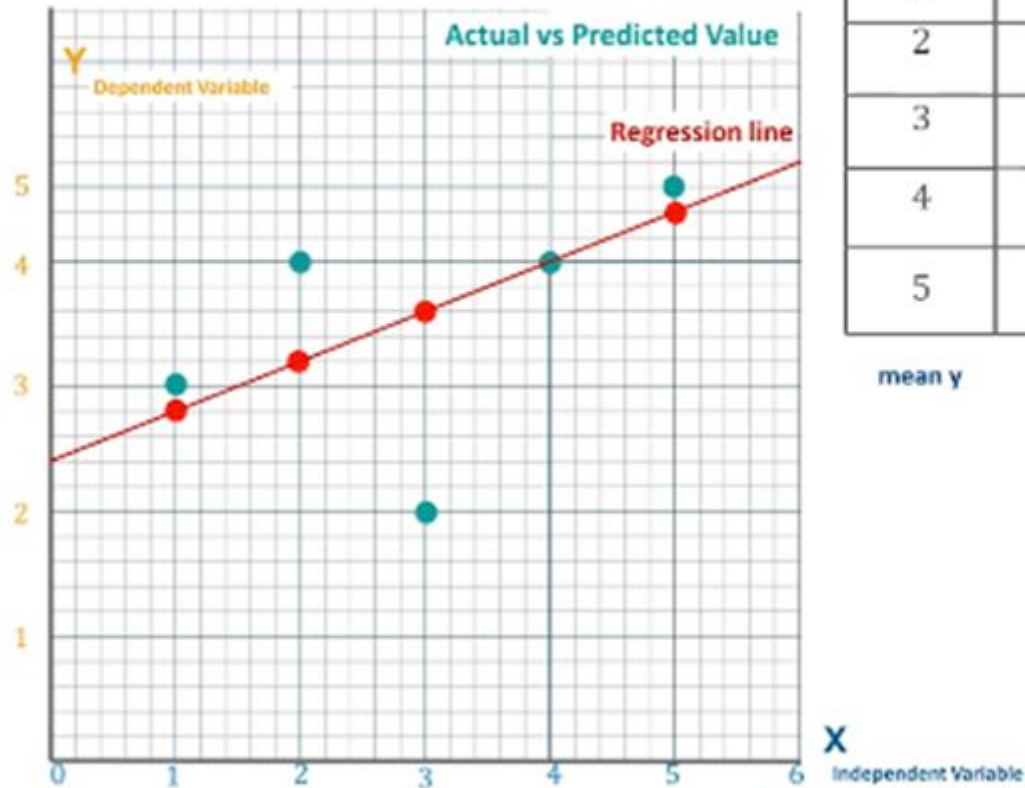
Distance actual - mean

vs

Distance predicted - mean

This is nothing but $R^2 = \frac{\sum (y_p - \bar{y})^2}{\sum (y - \bar{y})^2}$

GOODNESS OF FIT

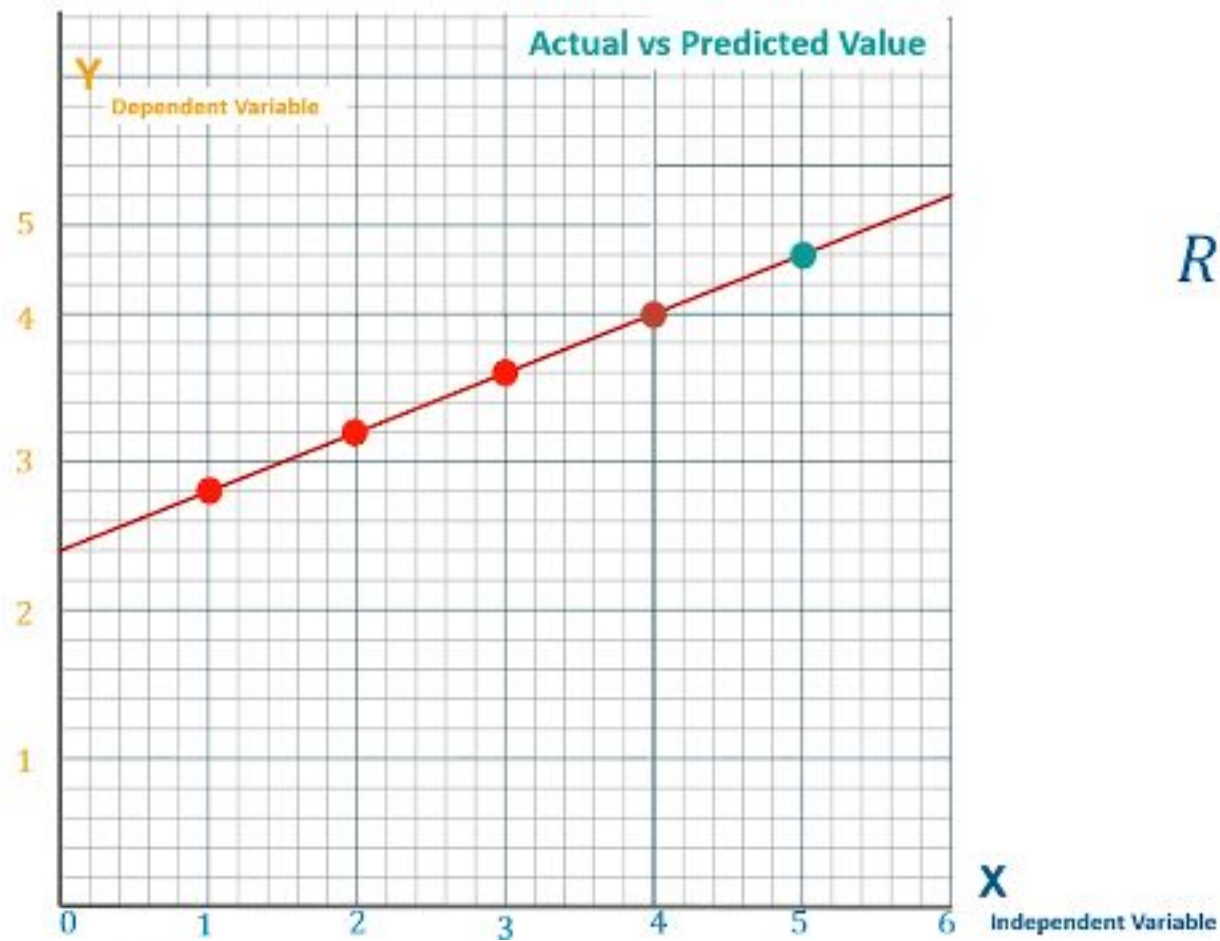


x	y	y - $\bar{y}$	(y - $\bar{y}$) ²	y_p	(y_p - y)	(y_p - y) ²
1	3	-0.6	0.36	2.8	-0.8	0.64
2	4	0.4	0.16	3.2	-0.4	0.16
3	2	-1.6	2.56	3.6	0	0
4	4	0.4	0.16	4.0	0.4	0.16
5	5	1.4	1.96	4.4	0.8	0.64
mean y		3.6	Σ 5.2		Σ 1.6	

$$R^2 = \frac{\Sigma(y_p - \bar{y})^2}{\Sigma(y - \bar{y})^2},$$

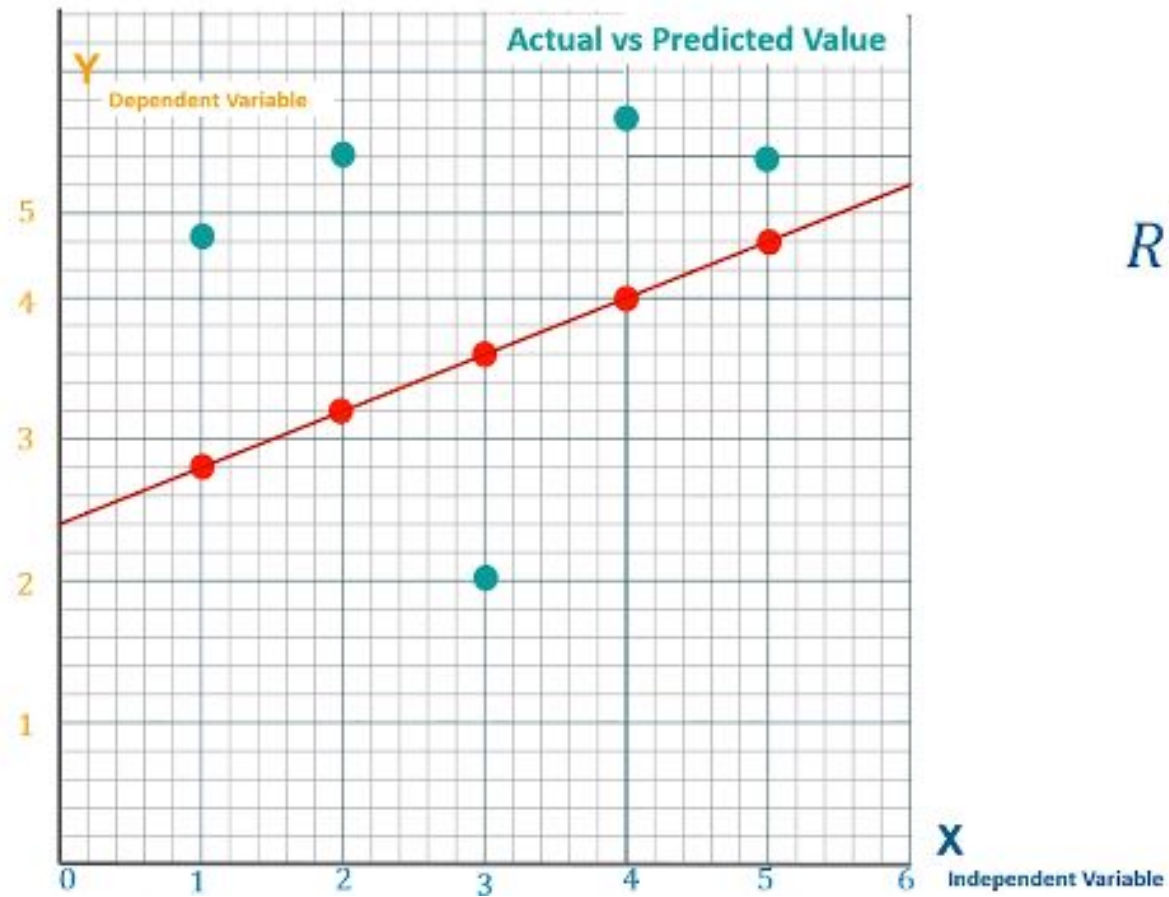
$$R^2 = \frac{1.6}{5.2}, R^2 = 0.3$$

GOODNESS OF FIT



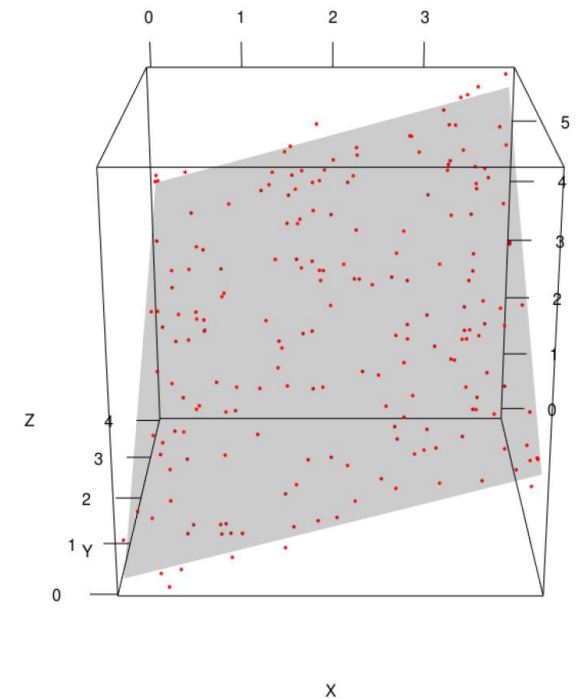
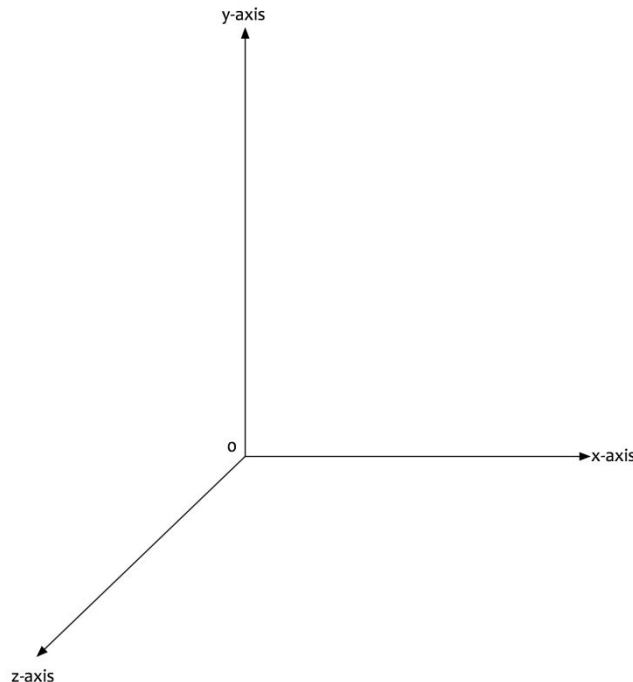
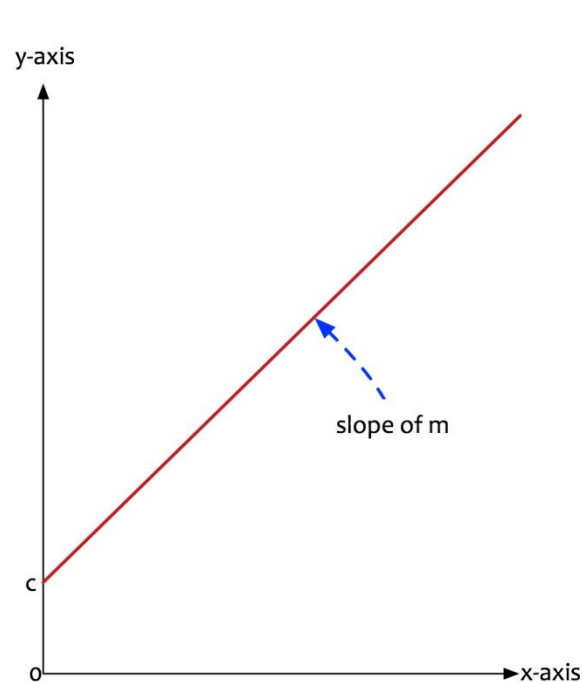
$$R^2 \approx 1$$

GOODNESS OF FIT



MULTIVARIATE LINEAR REGRESSION

More than one independent variables are used to predict the value of a numerical dependent variable.



MULTIVARIATE LINEAR REGRESSION

$$Y=f(x,z)$$

$$y = m1.x + m2.z + c$$

y is the dependent variable i.e. the variable that needs to be estimated and predicted.

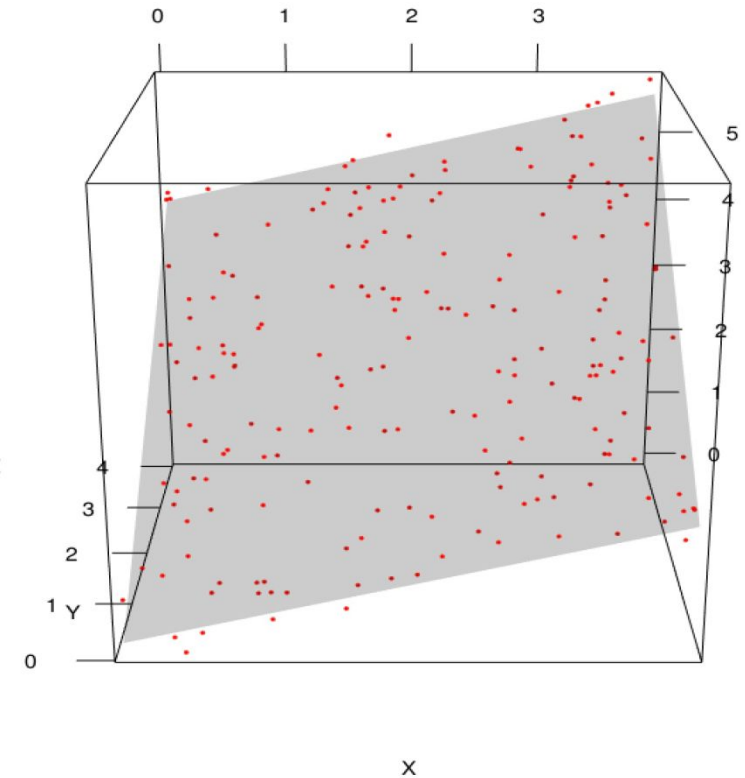
x is the first independent variable i.e. the variable that is controllable. It is the first input.

m1 is the slope of x1. It determines what will be the angle of the line (x).

z is the second independent variable i.e. the variable that is controllable. It is the second input.

m2 is the slope of z. It determines what will be the angle of the line (z).

c is the intercept. A constant that determines the value of y when x and z are 0.



MULTIVARIATE LINEAR REGRESSION

model with two input variables can be expressed as:

$$y = \beta_0 + \beta_1.x_1 + \beta_2.x_2$$

In machine learning world, there can be many dimensions. A model with three input variables can be expressed as:

$$y = \beta_0 + \beta_1.x_1 + \beta_2.x_2 + \beta_3.x_3$$

A generalized equation for the multivariate regression model can be:

$$y = \beta_0 + \beta_1.x_1 + \beta_2.x_2 + \dots + \beta_n.x_n$$

MULTIVARIATE LINEAR REGRESSION

Multiple features (variables).

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Notation:

n = number of features

$x^{(i)}$ = input (features) of i^{th} training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

When we have multiple features and we want to train a model that can predict the price given those features, we can use a multivariate linear regression. The model will have to learn the parameters(θ_0 to θ_n) on the training dataset below such that if we want to predict the price for a house that is not sold yet, it can give us prediction that is closer to what it will get sold for.

MULTIVARIATE LINEAR REGRESSION

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

For convenience of notation, define $x_0 = 1$. $(x_0^{(i)} = 1)$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$
$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \theta^T x$$
$$\left| \begin{array}{c} \underbrace{[\theta_0 \ \theta_1 \ \cdots \ \theta_n]}_{\theta^T} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \\ (n+1) \times 1 \text{ matrix} \end{array} \right| x$$

COST FUNCTION AND GRADIENT DESCENT FOR MULTIVARIATE LINEAR REGRESSION

Hypothesis:

$$h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Parameters: $\theta_1, \theta_2, \theta_3, \dots \dots \theta_n$

Cost Function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

GRADIENT DESCENT

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^m [h_{\Theta}(x_i) - y_i]^2$$

Predicted Value True Value

Gradient Descent

$$\Theta_j = \Theta_j - \alpha \frac{\partial}{\partial \Theta_j} J(\Theta)$$

Learning Rate

$$\begin{aligned} \frac{\partial}{\partial \Theta} J_{\Theta} &= \frac{\partial}{\partial \Theta} \frac{1}{2m} \sum_{i=1}^m [h_{\Theta}(x_i) - y]^2 \\ &= \frac{1}{m} \sum_{i=1}^m (h_{\Theta}(x_i) - y) \frac{\partial}{\partial \Theta_j} (\Theta x_i - y) \\ &= \frac{1}{m} \sum_{i=1}^m (h_{\Theta}(x_i) - y) x_i \end{aligned}$$

Therefore,

$$\Theta_j := \Theta_j - \frac{\alpha}{m} \sum_{i=1}^m [(h_{\Theta}(x_i) - y) x_i]$$

GRADIENT DESCENT

no of theta values = no of features + 1

repeat until convergence: {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_2^{(i)}$$

...

}

PRACTICAL IDEAS FOR MAKING GRADIENT DESCENT WORK WELL

Use feature scaling to help gradient descent converge faster. Get every feature between -1 and +1 range. It doesn't have to be exactly in the -1 to +1 range but it should be close to that range.

Feature Scaling

Idea: Make sure features are on a similar scale.

E.g. x_1 = size (0-2000 feet²)

x_2 = number of bedrooms (1-5)

$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$

$$0 \leq x_1 \leq 1 \quad 0 \leq x_2 \leq 1$$

PRACTICAL IDEAS FOR MAKING GRADIENT DESCENT WORK WELL

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean
(Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{\text{size} - 1000}{2000}$

$$x_2 = \frac{\#bedrooms - 2}{5}$$

$$-0.5 \leq x_1 \leq 0.5 \quad -0.5 \leq x_2 \leq 0.5$$

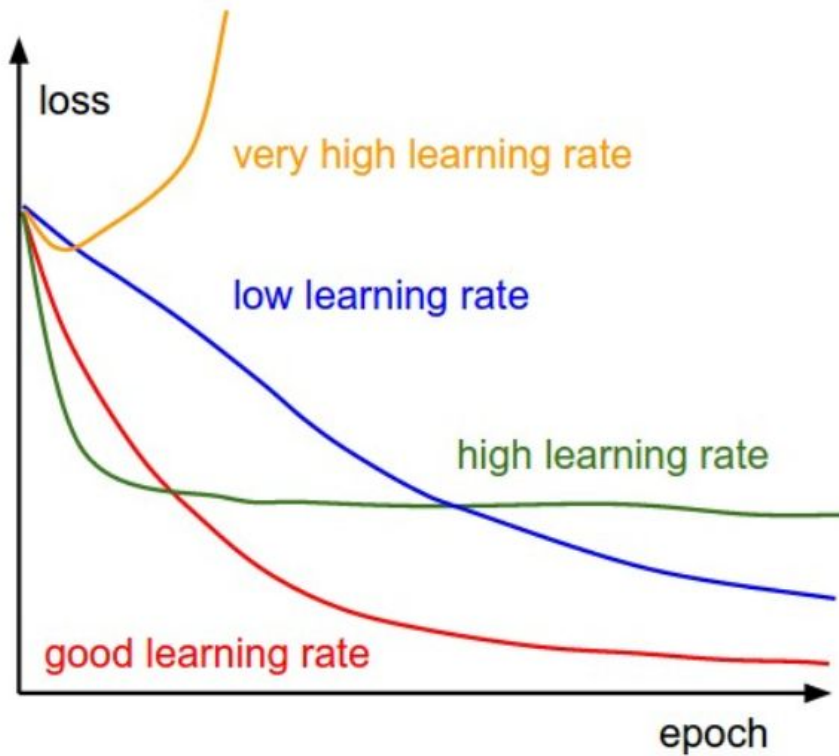
Handwritten formulas for feature normalization:

$$x_1 \leftarrow \frac{x_1 - \mu_1}{s_1} \quad \left| \quad x_2 \leftarrow \frac{x_2 - \mu_2}{s_2}$$

Annotations for x_1 :

- μ_1 : avg value of x_1 in training set
- s_1 : range (max-min) (or standard deviation)

PRACTICAL IDEAS FOR MAKING GRADIENT DESCENT WORK WELL.



If α is very small, it would take long time to converge and become computationally expensive.

If α is large, it may fail to converge and overshoot the minimum.

The most commonly used rates are : *0.001, 0.003, 0.01, 0.03, 0.1, 0.3*.

Gradient descent with different learning rates.

STEPS

STEP1: Initialize values $\beta_0, \beta_1, \dots, \beta_n$ with some value. In this case we will initialize with 0. Pick a value for the learning rate α .

STEP 2: Make sure to scale the features (X) if it's on a very different scales.

STEP 3: Iteratively update,

$$\beta_j := \beta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\beta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

The algorithm will converge when slope becomes

0

$$\beta_{j_new} = \beta_{j_old} - \alpha \times 0$$

$$\beta_{j_new} = \beta_{j_old}$$

MODEL INTERPRETATION

$$y = -85090 + 102.85 * X1 + 43.79 * X2 + 1.52 * x3 - 37.91 * x4 + 908.12 * x5 + 364.33 * x6$$

x1: With all other predictors held constant, if the x1 is increased by one unit, the average price **increases** by \$102.85.

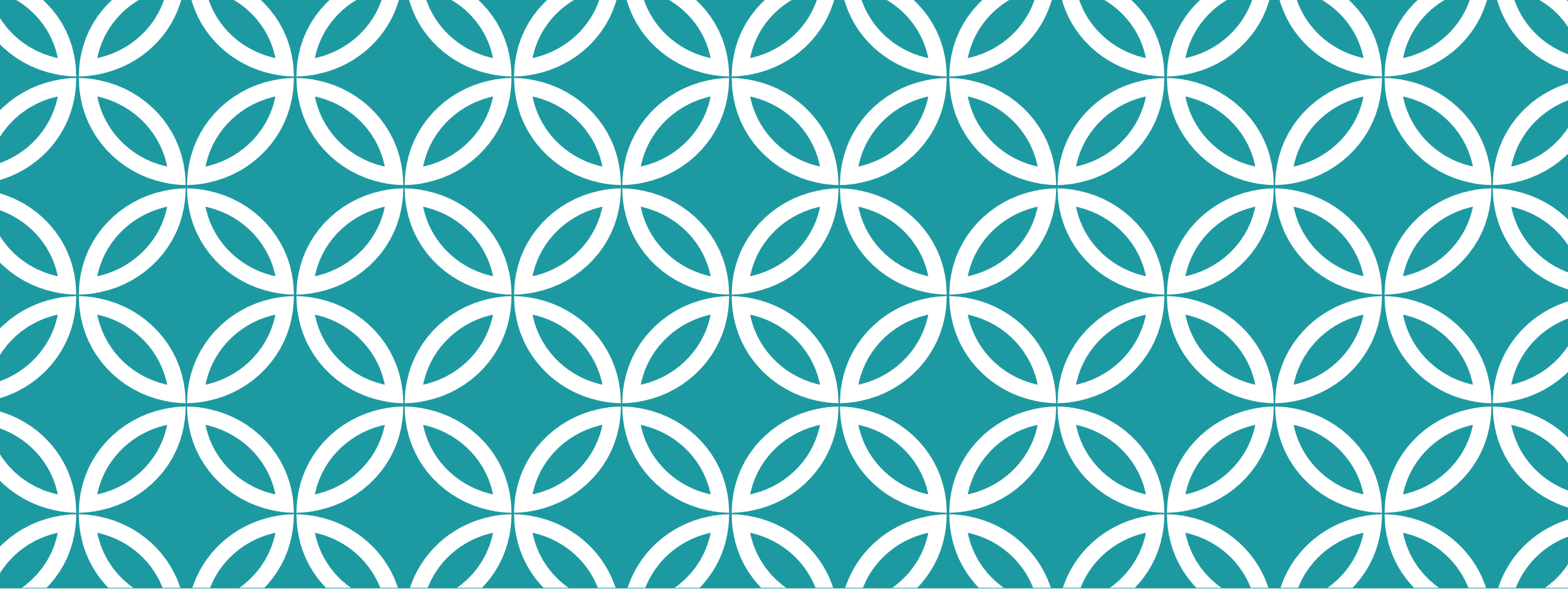
x2: With all other predictors held constant, if the x2 is increased by one unit, the average price **increases** by \$43.79.

x3: With all other predictors held constant, if the x3 is increased by one unit, the average price **increases** by \$1.52.

x4: With all other predictors held constant, if the x4 is increased by one unit, the average price **decreases** by \$37.91 (length has a -ve coefficient).

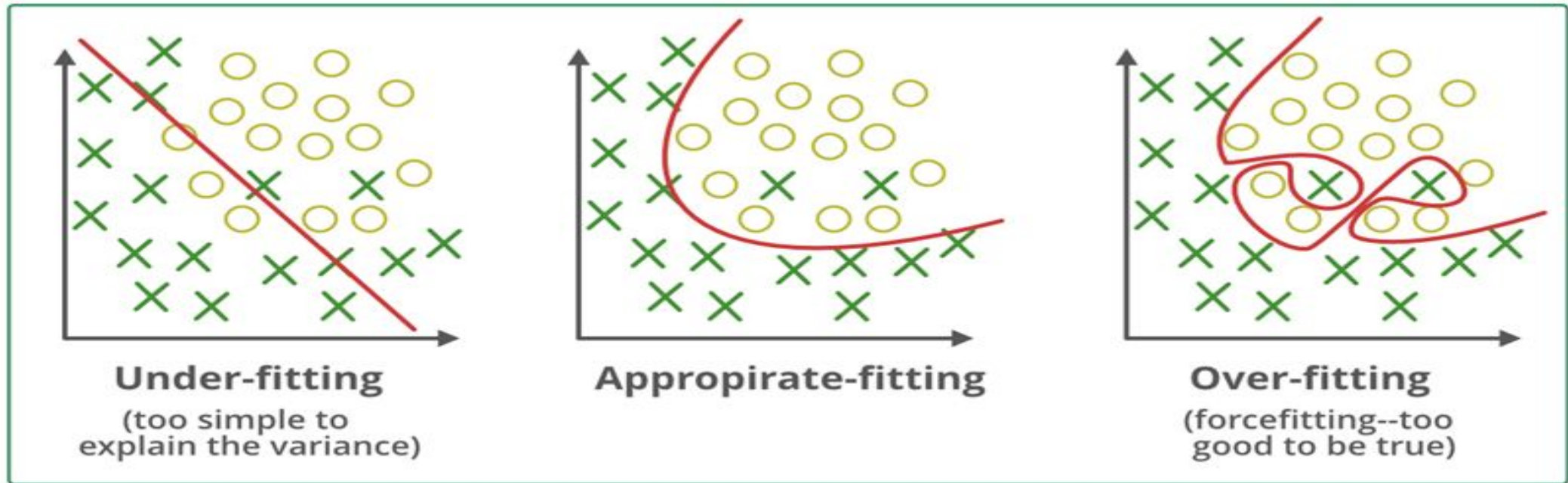
x5: With all other predictors held constant, if the x5 is increased by one unit, the average price **increases** by \$908.12

x6t: With all other predictors held constant, if the x6 is increased by one unit, the average price **increases** by \$364.33



REGULARIZATION

PROBLEM OF UNDER-FITTING AND OVER-FITTING

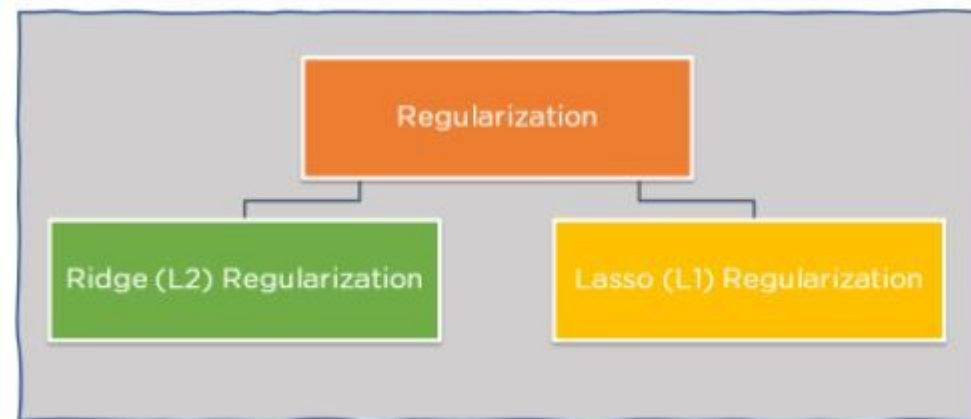


Regularisation is a technique used to reduce the errors by fitting the function appropriately on the given training set and avoid overfitting.

REGULARISATION IN ML

Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting.

Using Regularization, we can fit our machine learning model appropriately on a given test set and hence reduce the errors in it.



Note: LASSO (Least Absolute Shrinkage and Selection Operator)

RIDGE REGULARIZATION

It modifies the over-fitted or under fitted models by adding the penalty equivalent to the sum of the squares of the magnitude of coefficients.

Ridge Regression performs regularization by shrinking the coefficients present.

$$\text{Cost function} = \text{Loss} + \lambda \times \sum \|w\|^2$$

Here,

Loss = Sum of the squared residuals

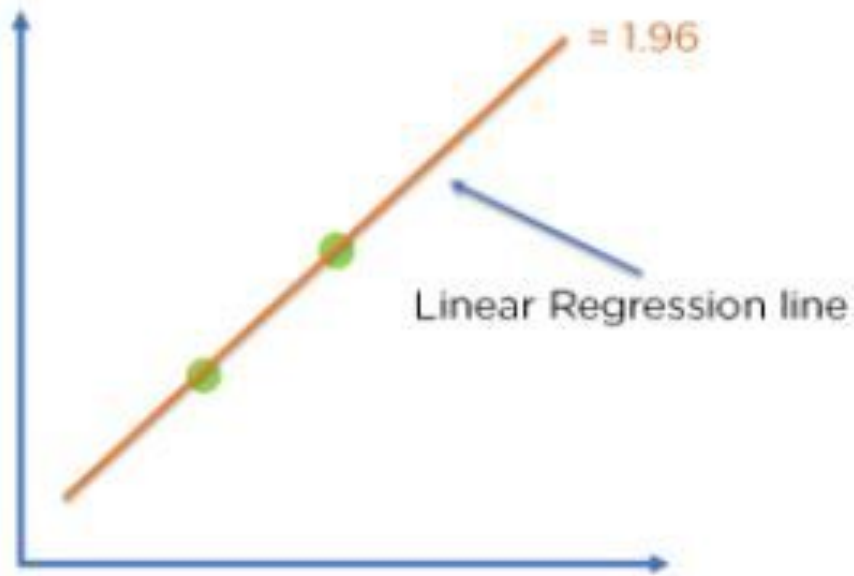
λ = Penalty for the errors

W = slope of the curve/ line

In the cost function, the penalty term is represented by Lambda λ . By changing the values of the penalty function, we are controlling the penalty term. The higher the penalty, it reduces the magnitude of coefficients. It shrinks the parameters. Therefore, it is used to prevent multicollinearity, and it reduces the model complexity by coefficient shrinkage.

RIDGE REGULARIZATION

Consider the graph illustrated below which represents Linear regression :



Cost function = Loss + $\lambda \times \sum \|w\|^2$

For Linear Regression line, let's consider two points that are on the line,

Loss = 0 (considering the two points on the line)

$\lambda = 1$

$w = 1.4$

Then, Cost function = $0 + 1 \times 1.42$
= 1.96

For Ridge Regression, let's assume,

Loss = $0.32 + 0.22 = 0.54$

$\lambda = 1$

$w = 0.7$

Then, Cost function = $0.54 + 1 \times 0.72$
= 1.26

RIDGE REGULARIZATION

Comparing the two models, with all data points, we can see that the Ridge regression line fits the model more accurately than the linear regression line.

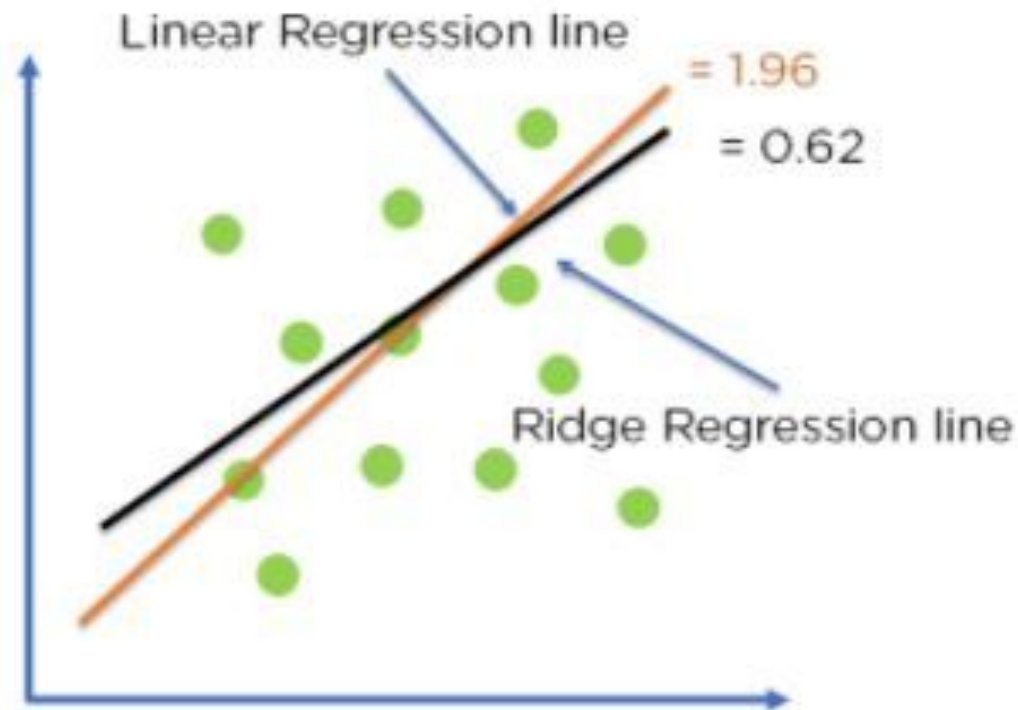


Figure: Optimization of model fit using Ridge

LASSO REGRESSION

It modifies the over-fitted or under-fitted models by adding the penalty equivalent to the sum of the absolute values of coefficients.

Lasso regression also performs coefficient minimization, but instead of squaring the magnitudes of the coefficients, it takes the true values of coefficients. This means that the coefficient sum can also be 0, because of the presence of negative coefficients

Cost function = Loss + $\lambda \times \sum \|w\|$

Here,

Loss = Sum of the squared residuals

λ = Penalty for the errors

w = slope of the curve/ line

We can control the coefficient values by controlling the penalty terms, just like we did in Ridge Regression.

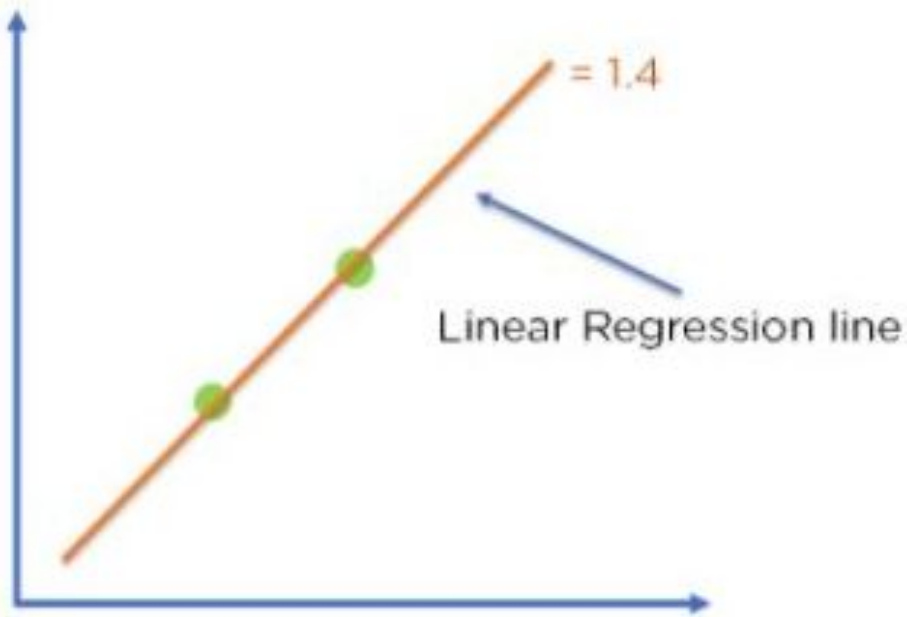


Figure: Linear Regression

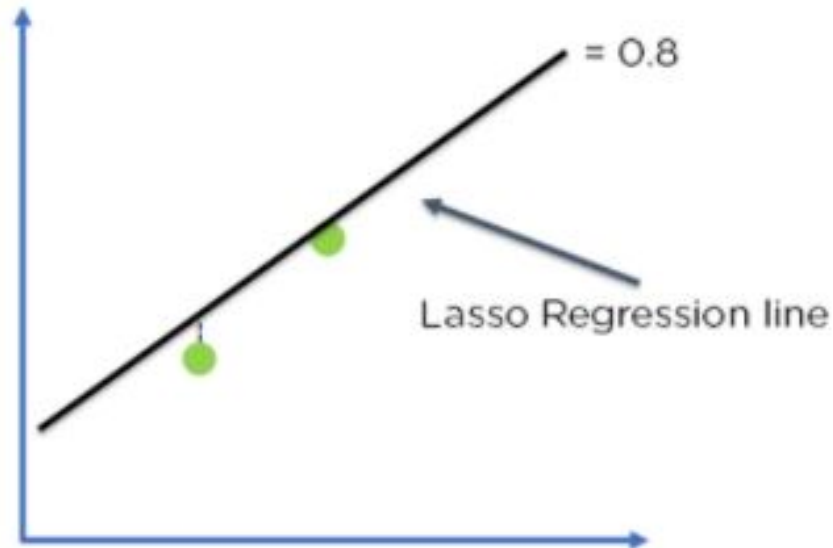


Figure 13: Lasso Regression

Cost function = Loss + $\lambda \times \sum \|w\|$

For Linear Regression line, let's assume,

Loss = 0 (considering the two points on the line)

$\lambda = 1$

$w = 1.4$

Then, Cost function = $0 + 1 \times 1.4$
 $= 1.4$

For Ridge Regression, let's assume,

Loss = $0.32 + 0.12 = 0.1$

$\lambda = 1$

$w = 0.7$

Then, Cost function = $0.1 + 1 \times 0.7$
 $= 0.8$

Comparing the two models, with all data points, we can see that the Lasso regression line fits the model more accurately than the linear regression line.